

# Project Design Phase-II

## Technology Stack (Architecture & Stack)

|               |                    |
|---------------|--------------------|
| Date          | 31 January 2025    |
| Team ID       | Individual Project |
| Project Name  | Book - A - Doctor  |
| Maximum Marks | 4 Marks            |

### Technical Architecture

The **Book A Doctor** application follows a **3-Tier Architecture** consisting of:

#### Presentation Layer

- React.js Frontend
- User Interface for Patients and Doctors

#### Business Logic Layer

- Node.js and Express.js Backend
- Authentication and Authorization
- Appointment Management
- Doctor Management
- Medical Report Management

#### Data Layer

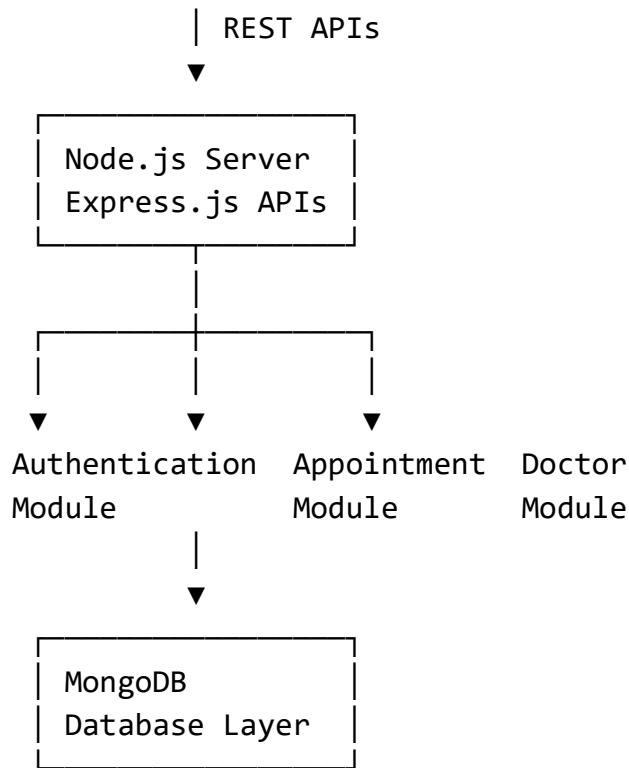
- MongoDB Database
- Storage of User, Doctor, Appointment, and Report Data

### Architecture Flow

Patients / Doctors



React.js UI  
Frontend Layer



**Table-1: Components & Technologies**

### 1. User Interface

**Description:** Web interface for patients and doctors to access the system.

**Technology:** React.js, HTML5, CSS3, JavaScript, Tailwind CSS

### 2. Application Logic-1

**Description:** User Authentication and Authorization.

**Technology:** Node.js, Express.js, JWT Authentication

### 3. Application Logic-2

**Description:** Appointment Booking and Management.

**Technology:** Node.js, Express.js

#### 4. Application Logic-3

**Description:** Doctor Management and Medical Report Handling.

**Technology:** Node.js, Express.js

#### 5. Database

**Description:** Stores user details, doctor profiles, appointments, and reports.

**Technology:** MongoDB, Mongoose

#### 6. Cloud Database

**Description:** Database hosting for production deployment.

**Technology:** MongoDB Atlas

#### 7. File Storage

**Description:** Storage of uploaded medical reports and documents.

**Technology:** Local File System / Cloud Storage

#### 8. External API-1

**Description:** Authentication and Email Notification Services.

**Technology:** Nodemailer API

#### 9. External API-2

**Description:** Future Appointment Reminder Notifications.

**Technology:** SMS Gateway API (Future Enhancement)

#### 10. Machine Learning Model

**Description:** Not implemented in the current version.

**Technology:** N/A

## 11. Infrastructure (Server / Cloud)

### Local Server Configuration:

- Windows/Linux Operating System
- Node.js Runtime Environment
- MongoDB Server
- VS Code Development Environment

### Cloud Server Configuration:

- Vercel (Frontend Deployment)
- Render/Railway (Backend Deployment)
- MongoDB Atlas (Database)

**Technology:** Local Server, Vercel, Render, MongoDB Atlas

## Table-2: Application Characteristics

### 1. Open-Source Frameworks

#### Description:

The application uses open-source frameworks for frontend and backend development.

#### Technology:

- React.js
- Node.js
- Express.js
- MongoDB
- Mongoose
- Tailwind CSS

### 2. Security Implementations

#### Description:

Security measures implemented to protect user data and application resources.

#### Technology:

- JWT Authentication
- Password Hashing using bcrypt
- Role-Based Access Control
- Secure REST APIs
- Input Validation

### 3. Scalable Architecture

#### Description:

The application follows a three-tier architecture that separates frontend, backend, and database components, allowing independent scaling.

#### Technology:

- React.js
- Node.js
- Express.js
- MongoDB Atlas

### 4. Availability

#### Description:

The application is accessible through web browsers and can be deployed on cloud infrastructure to ensure high availability.

#### Technology:

- Vercel
- Render
- MongoDB Atlas

### 5. Performance

#### Description:

The application is designed to provide fast response times for authentication, doctor search, appointment booking, and dashboard operations.

#### Technology:

- REST APIs

- MongoDB Indexing
- Efficient Database Queries
- React Component Rendering
- JWT Session Management